

LINUX PROJECTS FOR BEGINNERS

20 Hands-On Projects for
Junior DevOps and Linux Engineers



PRACTICAL
PROJECTS



REAL-WORLD
SKILLS



STEP-BY-STEP
GUIDES



ACTUAL
COMMANDS



BUILD YOUR
DEVOPS CAREER

- ✓ Server setup and management
- ✓ Users, permissions and security
- ✓ Automation with shell scripts and cron
- ✓ Systemd services
- ✓ Networking and troubleshooting
- ✓ Real-world troubleshooting scenarios
- ✓ A complete final project

PRACTICE
TODAY
A STRONGER
ENGINEER
TOMORROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD YOUR LINUX PRACTICE WORKSPACE



1 OPEN AND NAVIGATE THE TERMINAL

The terminal is your command line interface. It lets you interact with your Linux system using text commands.

BASIC COMMANDS

<code>pwd</code>	– Show current directory
<code>ls</code>	– List files and folders
<code>cd</code>	– Change directory
<code>clear</code>	– Clear the terminal

```
ubuntu@server:~$ pwd
/home/ubuntu
ubuntu@server:~$ ls
Desktop Documents Downloads
ubuntu@server:~$ cd Documents
ubuntu@server:~/Documents$ pwd
/home/ubuntu/Documents
ubuntu@server:~/Documents$ clear
ubuntu@server:~/Documents$
```



JUNIOR ENGINEER TIP

Get comfortable with the terminal. It's faster, more powerful and essential for real DevOps work.

2 CREATE A DEDICATED LINUX-LABS WORKSPACE

Keep your practice files in a separate folder so you stay organized and avoid mixing with your system files.

```
mkdir -p ~/linux-labs
cd ~/linux-labs
mkdir -p {p jects,notes,logs}
# This creates a clean workspace for your labs.
```

3 EXPLORE IMPORTANT SYSTEM DIRECTORIES

These directories contain useful system files and configurations.

<code>/home</code>	– User home directories
<code>/etc</code>	– System configuration files
<code>/var</code>	– Variable data (logs, databases, etc.)
<code>/tmp</code>	– Temporary files (safe to clear)

4 PROJECT GOAL



Create an organized lab environment.

- ✓ Set up your linux-labs folder
- ✓ Explore key system directories
- ✓ Be ready for the next projects



WHAT YOU WILL LEARN

- How to use the terminal
- Basic navigation commands
- How to create and organize folders
- Where important Linux directories are
- How to set up a clean workspace for labs



REMEMBER

A well-organized workspace makes you faster, safer and more productive.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



BUILD A PROJECT DIRECTORY FROM SCRATCH

Learn
Build
Practice
Grow

Organize your files like a real Linux engineer

1 KEY COMMANDS

COMMAND	WHAT IT DOES
<code>mkdir</code>	Create a new directory
<code>touch</code>	Create a new empty file
<code>cp</code>	Copy files or directories
<code>mv</code>	Move or rename files or directories
<code>rm</code>	Remove files or directories

2 RELATIVE vs ABSOLUTE PATHS

Absolute Path

Starts from the root (/)
Example: `/home/ubuntu/projects`
Always points to the same location.

Relative Path

Starts from your current directory
Example: `./projects` or `projects`
Depends on where you are.



JUNIOR ENGINEER TIP

Use absolute paths when you need certainty.
Use relative paths for quicker, shorter commands when you are already in the right location.

3 CREATE YOUR PROJECT STRUCTURE

Let's create a project called `linux-labs` with subdirectories for a clean and organized workspace.

```
# Go to your home directory (optional)
cd ~
# Create the main project directory
mkdir -p linux-labs
# Move into the project directory
cd linux-labs
# Create subdirectories
mkdir app config logs scripts backup
# Verify the structure
ls -l
```

4 MOVE AND RENAME FILES

Create a test file, move it, and rename it.

```
# Create a file in the current directory
touch app.txt
# Move the file to the app/ directory
mv app.txt app/
# Rename the file
mv app/app.txt app/myapp.txt
# Copy the file to the backup directory
cp app/myapp.txt backup/
# List files to confirm
ls app/ backup/
```

5 VERIFY THE FINISHED STRUCTURE

Use the `tree` command to view your directory structure.

```
# If tree is not installed (on Ubuntu/Debian)
sudo apt update && sudo apt install tree
# View the structure
tree linux-labs
```

```
ubuntu@server:~$ tree linux-labs
linux-labs/
├── app/
│   └── myapp.txt
├── backup/
│   └── myapp.txt
├── config/
├── logs/
└── scripts/

5 directories, 2 files
```

6 PROJECT GOAL



- ✓ Create an organized lab environment
- ✓ Practice essential Linux file commands
- ✓ Understand paths
- ✓ Manage files and directories confidently



REMEMBER

A well-organized directory structure makes you more productive, especially in real DevOps work.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta





CREATE AND MANAGE LINUX USERS

Learn
Build
Practice
Grow

Set up user accounts and groups like a Linux engineer

1 CREATE A PRACTICE USER

Create a new user account for practice.

```
# Create a new user
sudo useradd devuser

# Set a password for the user
sudo passwd devuser
```

Explanation:

- `useradd` creates a new user.
- `passwd` sets the user's password.

2 CREATE A GROUP

Create a group for your operations team.

```
# Create a new group
sudo groupadd devops

# Verify the group
getent group devops
```

Explanation:

- `groupadd` creates a new group.
- `getent` verifies the group exists.

3 ADD THE USER TO THE GROUP

Add the practice user to the group you created.

```
# Add user to group
sudo usermod -aG devops devuser

# Verify group membership
groups devuser
id devuser
```

Explanation:

- `usermod -aG` adds the user to the group.
- `groups` shows group membership.
- `id` displays user and group IDs.

4 INSPECT /etc/passwd

View the user entry in the system's user database.

```
# View all users
cat /etc/passwd

# Find your user
grep devuser /etc/passwd
```

Explanation:

- `/etc/passwd` stores user account information.
- Each line represents a user.

5 USE id, groups, and whoami

Check user details and current identity.

```
# Show user ID and groups
id devuser

# Show groups for the user
groups devuser

# See who you are logged in as
whoami
```

Explanation:

- `id` shows user and group IDs.
- `groups` shows group membership.
- `whoami` shows the current logged-in user.

7 SECURITY LESSON



NEVER SHARE ADMINISTRATOR ACCOUNTS

- Each person should have their own user account.
- Use groups to manage permissions.
- Only use `sudo` when necessary.
- Shared accounts make it impossible to track activity and increase security risks.
- Good user and group management is a basic requirement in real production environments.



JUNIOR ENGINEER TIP

Use meaningful usernames (e.g. `alice`, `bob`, `devops`) and groups to keep your system organized from the start.

6 PROJECT: PREPARE ACCOUNTS FOR A SMALL OPERATIONS TEAM

Create multiple users and groups for a small team.

```
# Create groups
sudo groupadd devops
sudo groupadd sre
sudo groupadd ops

# Create users
sudo useradd alice
sudo useradd bob
sudo useradd charlie

# Set passwords (run for each user)
sudo passwd alice

# Add users to groups
sudo usermod -aG devops alice
sudo usermod -aG sre bob
sudo usermod -aG ops charlie
```

Your Task:

1. Create 3 users.
2. Create 3 groups.
3. Add each user to the appropriate group.
4. Verify everything works using `id` and `groups`.
5. Document the users and their roles.



Real teams use separate accounts for better security, tracking and accountability.



PROJECT COMPLETE!

You now know how to create users, manage groups, inspect system files, and prepare accounts for a team.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta





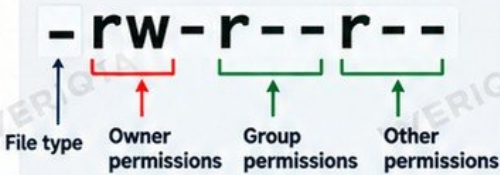
SECURE A SENSITIVE CONFIGURATION FILE

Learn
Build
Practice
Grow

Understand and apply ownership and permissions like a Linux engineer

1 OWNERSHIP vs PERMISSIONS

Every file in Linux has an owner, a group and permissions that control who can read (r), write (w) and execute (x) it.



r = read
w = write
x = execute
- = no permission

2 VIEW FILE DETAILS WITH ls -l

Use `ls -l` to see the owner, group and permissions.

```
ubuntu@server:~$ ls -l /etc/nginx/nginx.conf
-rw-r--r-- 1 root root 1234 Aug 20 10:15
/etc/nginx/nginx.conf
```

Explanation:

- `-rw-r--r--` → file permissions
- `1` → number of hard links
- `root` → file owner
- `root` → group owner
- `1234` → file size in bytes
- `Aug 20 10:15` → last modified date and time
- `/etc/nginx/nginx.conf` → file path

3 CHANGE PERMISSIONS WITH chmod

Use `chmod` to change file permissions.

```
ubuntu@server:~$ chmod 600 /etc/myapp/config.conf
ubuntu@server:~$ ls -l /etc/myapp/config.conf
-rw----- 1 root root 512 Aug 20 11:00
/etc/myapp/config.conf
```

Common Examples:

- `chmod 600` → owner read/write only
- `chmod 644` → owner read/write, others read
- `chmod 755` → owner full, others read/execute

4 CHANGE OWNERSHIP WITH chown

Use `chown` to change the file owner.

```
ubuntu@server:~$ chown appuser:appgroup \
/etc/myapp/config.conf
ubuntu@server:~$ ls -l /etc/myapp/config.conf
-rw----- 1 appuser appgroup 512 Aug 20 11:05
/etc/myapp/config.conf
```

Syntax:

`chown user:group <file>`

Example:

`chown appuser:appgroup /etc/myapp/config.conf`

5 CHANGE GROUP WITH chgrp

Use `chgrp` to change the group owner.

```
ubuntu@server:~$ chgrp appgroup
/etc/myapp/config.conf
ubuntu@server:~$ ls -l /etc/myapp/config.conf
-rw----- 1 appuser appgroup 512 Aug 20 11:05
/etc/myapp/config.conf
```

6 SECURE AN APPLICATION CONFIG FILE

Let's secure a sensitive configuration file for an application.

```
# 1. Create a sample config file
echo "DB_PASSWORD=secret123" > app.conf
# 2. Set proper ownership and permissions
chown appuser:appgroup app.conf
chmod 600 app.conf
# 3. Verify
ls -l app.conf
-rw----- 1 appuser appgroup 20 Aug 20 11:10 app.conf
```

Result:

- ✓ File is owned by the application user
- ✓ Only the owner can read and write
- ✓ The file is protected from other users

7 TEST ACCESS AS ANOTHER USER

Switch to another user and try to access the file.

```
ubuntu@server:~$ sudo -u bob cat app.conf
cat: app.conf: Permission denied
ubuntu@server:~$ sudo -u appuser cat app.conf
DB_PASSWORD=secret123
```

What this shows:

- ✓ User bob cannot read the file
- ✓ Only the owner (appuser) can read it

8 BEGINNER MISTAKE: USING chmod 777



Do NOT use `chmod 777` on sensitive files.

```
ubuntu@server:~$ chmod 777 app.conf
ubuntu@server:~$ ls -l app.conf
-rwxrwxrwx 1 appuser appgroup 20 Aug 20 11:15 app.conf
```

Why this is a problem:

- ✗ Gives full access to everyone
- ✗ Anyone can read, modify or delete the file
- ✗ High security risk, especially for sensitive data like passwords, API keys and configuration files.

★ KEY TAKEAWAYS

- ✓ Understand file ownership and permissions
- ✓ Use `ls -l` to inspect files
- ✓ Use `chmod`, `chown` and `chgrp` correctly
- ✓ Always secure sensitive configuration files
- ✓ Test access as another user
- ✓ Never use `chmod 777` on sensitive files



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta





PROTECT AN SSH PRIVATE KEY

Keep your keys safe, keep your servers safe

Learn
Build
Practice
Grow

1 CREATE A SIMULATED SSH KEY FILE

Let's create a dummy SSH private key file for practice.

```
ubuntu@server:~$ mkdir -p ~/ssh-labs
ubuntu@server:~$ cd ~/ssh-labs
ubuntu@server:~/ssh-labs$ touch id_rsa
ubuntu@server:~/ssh-labs$ echo "-----BEGIN
OPENSsh PRIVATE KEY-----" > id_rsa
ubuntu@server:~/ssh-labs$ echo "(simulated key)"
>> id_rsa
ubuntu@server:~/ssh-labs$ echo "-----END OPENSsh
PRIVATE KEY-----" >> id_rsa
```

What this does:

- Creates a practice directory
- Creates a fake private key file
- Adds sample content (not a real key)

2 WHY PRIVATE KEYS NEED RESTRICTED PERMISSIONS

An SSH private key gives full access to your servers. If someone else can read this file, they can log in as you.



KEY RISKS

- Can be used to access your servers
- Can be copied and misused
- May give access to production systems
- Often targeted by attackers



BEST PRACTICE

- Keep your private keys secure
- Restrict file permissions (only you can read it)
- Store keys in a safe location
- Never share your private key

3 SET THE CORRECT PERMISSIONS (chmod 600)

Use **chmod 600** to allow only the owner to read and write the file.

```
ubuntu@server:~/ssh-labs$ chmod 600 id_rsa
ubuntu@server:~/ssh-labs$ ls -l id_rsa
-rw----- 1 ubuntu ubuntu 78 Aug 20 11:15
id_rsa
```

What 600 means:

- 6 (owner) = read + write
- 0 (group) = no permission
- 0 (others) = no permission
- Only you can read and write the file

4 INSPECT PERMISSIONS

Use **ls -l** to check the file permissions.

```
ubuntu@server:~/ssh-labs$ ls -l id_rsa
-rw----- 1 ubuntu ubuntu 78 Aug 20 11:15 id_rsa
```

How to read the output:

- **-rw-----** → file permissions
- **1** → number of hard links
- **ubuntu** → file owner
- **ubuntu** → group owner
- **78** → file size in bytes
- **Aug 20 11:15** → last modified date and time
- **id_rsa** → file name

5 COMPARE SECURE AND INSECURE PERMISSIONS

See the difference between correct and incorrect permissions.

Permission	Command	Result	Security Status
600	chmod 600 id_rsa	-rw-----	✓ Secure (Recommended)
644	chmod 644 id_rsa	-rw-r--r--	✗ Insecure (Readable by others)
777	chmod 777 id_rsa	-rwxrwxrwx	✗ Very Insecure (Everyone can read, modify and execute)

6 CONNECT TO SSH AUTHENTICATION

Your private key is used to authenticate with remote servers.

```
ssh -i id_rsa ubuntu@your-server-ip
```

What happens:

- The SSH client uses your private key to prove your identity
- The server verifies the key
- You get secure access without a password (if key is authorized)



NEVER DO THIS

- Do not use chmod 777 on private keys
- Do not share your private key with anyone
- Do not store keys in public or shared directories



KEY TAKEAWAYS

- Private keys must be protected
- Use chmod 600
- Always check permissions
- Always check permissions
- Insecure permissions can lead to unauthorized access
- A private key is the key to your servers — keep it safe



FIND FILES LIKE A LINUX ENGINEER

Find what you need, faster and smarter

1 FIND with find

The **find** command searches for files and directories based on different criteria.

```
ubuntu@server:~$ find
/etc -name "nginx.conf"
/etc/nginx/nginx.conf
/etc/nginx/conf.d/nginx.conf
```

Common Options:

- name "filename"
- type f (files)
- type d (directories)
- size +10M (larger than 10MB)
- mtime -7 (modified in last 7 days)
- user username
- path "/etc/*"

2 FIND with locate

The **locate** command searches a prebuilt database of files. It is fast but may not show recently created files (update the database if needed).

```
ubuntu@server:~$ locate nginx.conf
/etc/nginx/nginx.conf
/usr/share/doc/nginx/nginx.conf.gz
```

Update the database:
sudo updatedb



TIP

Use locate for quick searches, but use find for the most accurate and up-to-date results.

3 FIND with which

The **which** command shows the full path of a command (executable) in your PATH.

```
ubuntu@server:~$ which python3
/usr/bin/python3
```

Examples:

- which nginx
- which docker
- which kubectl

4 FIND with whereis

The **whereis** command finds the location of binaries, source files and manual pages.

```
ubuntu@server:~$ whereis nginx
nginx: /usr/sbin/nginx /etc/nginx
/usr/share/man/man8/nginx.8.gz
```

5 SEARCH BY FILENAME

Find files with a specific name.

```
# Search for a file by exact name
find / -name "app.conf" 2>/dev/null

# Case insensitive search
find / -iname "app.conf" 2>/dev/null
```

6 SEARCH BY EXTENSION

Find files with a specific file extension.

```
# Find all .conf files
find /etc -type f -name "*.conf"

# Find all .log files
find /var/log -type f -name "*.log"

# Find all .sh files
find /home -type f -name "*.sh"
```

7 SEARCH BY SIZE AND MODIFICATION TIME

Find files based on size and when they were last modified.

```
# Files larger than 100MB
find / -type f -size +100M

# Files smaller than 1MB
find / -type f -size -1M

# Files modified in last 7 days
find / -type f -mtime -7
```

8 PROJECT: LOCATE A MISSING CONFIGURATION FILE

Your task is to find a missing configuration file in a server directory. Use different commands and criteria to locate it.

Steps:

- Search for a file named app.conf
- If not found, search by extension (.conf)
- Look in common directories (/etc, /opt, /home)
- Check file size and modification time
- Verify the correct file and its contents

Example:

```
find / -name "app.conf" 2>/dev/null
find / -type f -name "*.conf" 2>/dev/null
find /etc -name "*.conf" 2>/dev/null
find / -type f -size +10M 2>/dev/null
find / -type f -mtime -7 2>/dev/null
```



PROJECT GOAL

Locate the missing configuration file and confirm it is the correct file.



JUNIOR ENGINEER TIP

Learn multiple ways to find files. In real environments, you may not know the exact name, location or when the file was created.



COMMON MISTAKES

- Using locate without updating the database
- Searching the wrong directory
- Not using sudo (permission denied)
- Assuming the first result is the correct file



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



INVESTIGATE APPLICATION LOGS

Find Problems. Understand What Happened. Fix It.

Learn
Practice
Build
Grow

1 BUILD A SAMPLE APP.LOG

Let's create a sample log file with real application-like logs.

```
mkdir -p ~/linux-labs/logs
cd ~/linux-labs/logs
cat > app.log << EOF
2024-08-25 10:00:01 INFO Application started
2024-08-25 10:01:15 INFO User login successful
2024-08-25 10:02:33 WARNING Low disk space
2024-08-25 10:03:21 ERROR Failed to connect to DB
2024-08-25 10:04:10 INFO Retrying connection
2024-08-25 10:05:42 ERROR Database unavailable
2024-08-25 10:06:18 INFO User request failed: 500
2024-08-25 10:07:33 INFO Application stopped
EOF
```

2 USEFUL LOG COMMANDS

Command	What It Does
<code>cat app.log</code>	Displays the entire log file
<code>less app.log</code>	View logs page by page (press q to exit)
<code>head app.log</code>	Show the first 10 lines
<code>tail app.log</code>	Show the last 10 lines
<code>tail -f app.log</code>	Follow the log in real time (use Ctrl+C to stop)
<code>grep ERROR app.log</code>	Search for lines containing ERROR
<code>grep WARNING app.log</code>	Search for lines containing WARNING
<code>grep " 500" app.log</code>	Search for failed requests (HTTP 500)

3 EXAMPLE OUTPUT

```
ubuntu@server:~/linux-labs/logs$ grep ERROR app.log
2024-08-25 10:03:21 ERROR Failed to connect to DB
2024-08-25 10:05:42 ERROR Database unavailable
ubuntu@server:~/linux-labs/logs$ tail -n 3 app.log
2024-08-25 10:05:42 ERROR Database unavailable
2024-08-25 10:06:18 INFO User request failed: 500
2024-08-25 10:07:33 INFO Application stopped
```

4 SEARCH FOR SPECIFIC ISSUES

What to Find	Command
All errors	<code>grep ERROR app.log</code>
All warnings	<code>grep WARNING app.log</code>
Failed requests (500)	<code>grep " 500" app.log</code>
Failed logins	<code>grep -i "failed" app.log</code>
Count errors	<code>grep ERROR app.log wc -l</code>

5 PROJECT: IDENTIFY WHY THE APPLICATION FAILED



- 1 Use the commands above to investigate the log file.
- 2 Find the main error.
- 3 Check what happened before the error.
- 4 Look for related warnings or failed requests.
- 5 Write a short summary of the root cause.



JUNIOR ENGINEER TIP

Logs tell a story. Look at the bigger picture, not just one error line. Check what happened before the error and after the error.



EXPECTED FINDING

In this example, the application failed because it could not connect to the database (Database unavailable), which caused user requests to fail with HTTP 500.



REMEMBER

In real systems, logs can be very large. Use tools like `less`, `grep`, and `tail -f` to find what you need quickly.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

MONITOR LINUX PROCESSES

Find. Analyze. Manage. Keep Your System Healthy.

Learn
Practice
Build
Grow

1 START A TEST PROCESS

Let's start a simple test process so we can find and manage it.

```
# Start a process that runs for a long time
sleep 300 &

# Check it's running
ps

# Or start a test script
while true; do echo "Linux is running...";
sleep 5; done &
```

The `&` runs the process in the background so you can continue using the terminal.

2 VIEW RUNNING PROCESSES

Use these commands to see what's running on your system.

Command	What It Does
<code>ps</code>	Show your current shell's processes
<code>ps aux</code>	Show all running processes (all users, full details)
<code>top</code>	Real-time process monitor (press <code>q</code> to exit)
<code>pgrep <name></code>	Find the PID of a process by name
<code>pidof <name></code>	Show PID(s) of a process
<code>htop</code>	Interactive process viewer (if installed)

3 UNDERSTAND PID (PROCESS ID)

Every running process has a unique Process ID (PID). You use the PID to manage (e.g. stop) the process.

```
ubuntu@server:~$ ps aux | grep sleep
ubuntu 12345 0.0 0.0 2880 892 pts/0 S 10:15 0:00 sleep 300
ubuntu 0.0 0.0 2880 896 pts/0 S 10:15 0:00 grep sleep
```

This is the PID (Process ID)

4 FIND A PROCESS BY NAME

Use `pgrep` or `grep` to search for a process.

```
ubuntu@server:~$ pgrep sleep
12345
ubuntu@server:~$ ps aux | grep nginx
root 543 0.0 0.1 7228 3120 ? Ss 10:00 0:00 nginx: master process
www-data 544 0.0 0.1 7380 3200 ? S 10:00 0:00 nginx: worker process
ubuntu 5678 0.0 0.0 2880 900 pts/0 S 10:20 0:00 grep nginx
```

5 KILL A PROCESS

Use the `kill` command with the PID.

Command	What It Does
<code>kill <PID></code>	Sends a normal termination signal (SIGTERM)
<code>kill -9 <PID></code>	Forces the process to stop (SIGKILL)
<code>pkill <name></code>	Kills a process by name
<code>killall <name></code>	Kills all processes with the name



Be careful before using `kill -9`. Only use it if the process does not stop with the normal kill command.

6 MONITOR WITH top

The `top` command shows real-time system activity.

```
top - 10:22:17 up 1:03, 1 user, load average: 0.10, 0.05, 0.01
Tasks: 214 total, 1 running, 213 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.2 us, 0.3 sy, 0.0 ni, 98.1 id, 0.0 wa, 0.0 hi, 0.4 si, 0.0 st
MiB Mem : 3927.1 total, 1024.5 free, 1450.3 used, 1452.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2340.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
12345	ubuntu	20	0	2880	892	700	S	0.0	0.0	0:00.00	sleep
2345	ubuntu	20	0	1620	412	320	R	0.7	0.1	0:00.12	top
1	root	20	0	167548	11852	8200	S	0.0	0.3	0:03.21	systemd

Press `q` to exit top.

7 PROJECT: LOCATE AND TERMINATE A PROBLEMATIC PROCESS



- 1 Start a test process (e.g. `sleep 300`).
- 2 Find the process using `ps` or `pgrep`.
- 3 Analyze the process details.
- 4 Terminate the process using `kill`.
- 5 Verify that the process has stopped.



JUNIOR ENGINEER TIP

Not all processes are safe to kill. Understand the process, check its purpose, and make sure it's not a critical system service.

8 KEY TAKEAWAYS

- ✓ Every process has a unique PID.
- ✓ Use `ps`, `top`, and `pgrep` to find processes.
- ✓ Use `kill` to stop a process safely.
- ✓ Understand what a process does before killing it.
- ✓ Monitoring processes helps you keep your system healthy.



WARNING

Killing the wrong process can break applications or the entire system. When in doubt, investigate first.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



DIAGNOSE CPU AND MEMORY USAGE

Learn
Practice
Build
Grow

Find Bottlenecks. Identify the Real Problem. Keep Your Server Healthy.

1 CHECK SYSTEM UPTIME AND LOAD

The **uptime** command shows how long the server has been running and the current load average.

```
ubuntu@server:~$ uptime
10:24:17 up 5 days, 3:12, 2 users,
load average: 1.25, 0.98, 0.76
```

Load average

- Three numbers show average CPU load over: 1 minute, 5 minutes, 15 minutes
- If the load average is higher than the number of CPU cores, the server is under heavy load.

2 CHECK MEMORY USAGE

The **free -h** command shows memory usage in a human-readable format.

```
ubuntu@server:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:            7.7Gi      5.1Gi      1.2Gi      512Mi      1.4Gi      2.1Gi
Swap:           2.0Gi      256Mi      1.8Gi
```

Key things to look at:

- used – memory currently in use
- free – actual free memory
- buff/cache – used for caching (normal)
- available – memory available for new apps
- Swap – used when RAM is full

3 MONITOR IN REAL TIME WITH top

The **top** command shows live CPU and memory usage by process. Press q to exit.

```
ubuntu@server:~$ top
top - 10:25:33 up 5 days, 3:13, 2 users, load average: 1.25, 0.98, 0.76
Tasks: 215 total, 1 running, 214 sleeping, 0 stopped, 0 zombie
%Cpu(s): 12.3 us, 5.1 sy, 0.0 ni, 82.1 id, 0.3 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 7871.5 total, 5234.1 used, 1234.5 free, 1402.9 buff/cache
MiB Swap: 2048.0 total, 256.0 used, 1792.0 free
```

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM	TIME+	COMMAND
1234	ubuntu	20	0	892M	120M	25.6	1.5	0:05:23	python
5678	ubuntu	20	0	412M	98M	12.3	1.2	0:02:11	nginx
9012	root	20	0	234M	45M	5.1	0.6	0:00:08	systemd

top shows which processes are using the most CPU and memory right now.

4 USE ps TO CHECK RUNNING PROCESSES

The **ps** command shows a snapshot of running processes.

```
ubuntu@server:~$ ps aux --sort=-%cpu | head -n 5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ubuntu    1234 25.6  1.5 892000 10000 ?        S    10:20   0:05 python
ubuntu    5678 12.3  1.2 412000  9000 ?        S    10:22   0:02 nginx
root      9012  5.1  0.6 234000  4500 ?        S    10:18   0:00 systemd
ubuntu    3456  2.3  0.4 120000  3000 ?        S    10:19   0:00 node
mysql     7890  1.8  0.3 512000 25000 ?        S    10:21   0:00 mysql
```

Useful options:

- ps aux – show all processes
- sort=-%cpu – sort by highest CPU usage
- sort=-%mem – sort by highest memory usage

5 UNDERSTAND SWAP USAGE

Swap is used when physical memory (RAM) is full. It's slower than RAM.

```
ubuntu@server:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:            7.7Gi      5.1Gi      1.2Gi      512Mi      1.4Gi      2.1Gi
Swap:           2.0Gi      256Mi      1.8Gi
```

If swap usage is high, it can indicate memory pressure. Occasional swap usage is normal, but constant high swap usage can slow down the server.

6 KEY METRICS TO MONITOR

- ✓ Uptime – how long the server has been running
- ✓ Load average – CPU demand over time
- ✓ CPU usage – which processes are using the CPU
- ✓ Memory usage – used, free, available
- ✓ Swap usage – is the server swapping?
- ✓ Running processes – identify resource hungry processes



TIP

High CPU or memory usage can be caused by a real workload, a misconfigured application, or even a runaway process.

7 PROJECT: IDENTIFY THE RESOURCE HUNGRY PROCESS



- 1 Run a test process (e.g. a CPU or memory intensive command).
- 2 Use top or ps to find which process is consuming the most resources.
- 3 Check CPU and memory usage.
- 4 Investigate what the process is doing.
- 5 Decide if it should be stopped.
- 6 Safely terminate the process if needed.

8 TROUBLESHOOTING QUESTION



Is the server actually out of memory?

- Check **free -h** to see available memory.
- Look at buff/cache (this is normal).
- Check swap usage.
- Identify which process is using the memory.
- Consider if the application has a memory leak.



WARNING

Understand a process before killing it. Some processes are critical for the system. Killing the wrong process can break applications or even make the server unstable.



JUNIOR ENGINEER TIP

High resource usage is a symptom. Always investigate the root cause, not just kill the process.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



INVESTIGATE A FULL DISK



Learn
Practice
Build
Grow

Find What's Using Space. Fix the Real Problem. Keep Your Server Healthy.

1 CHECK FILESYSTEM USAGE WITH `df -h`

The `df -h` command shows disk usage for all mounted filesystems.

```
ubuntu@server:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            7.8G     0  7.8G   0% /dev
tmpfs           1.6G   12M   1.6G   1% /run
/dev/sda1       50G   48G   2.0G  96% /
tmpfs           7.8G     0  7.8G   0% /dev/shm
/dev/sdb1      200G  120G   80G   60% /data
```

- **Size** – total size of the filesystem
- **Used** – space used
- **Avail** – available space
- **Use%** – percentage used
- **Mounted on** – mount point

If Use% is 90% or higher, your disk is almost full.

2 CHECK DIRECTORY SIZE WITH `du -sh`

The `du -sh` command shows the total size of a directory in a human readable format.

```
ubuntu@server:~$ du -sh /var/log
2.4G    /var/log
ubuntu@server:~$ du -sh /home
1.1G    /home
ubuntu@server:~$ du -sh /var/www
850M    /var/www
```

Use `du -sh` on specific directories to quickly see which ones are using the most space.

3 FIND LARGE FILES AND DIRECTORIES WITH `du`

Use `du` to explore which subdirectories are consuming space.

```
ubuntu@server:~$ du -h /var | sort -hr | head -10
2.4G    /var/log
1.1G    /var/lib
850M    /var/cache
420M    /var/www
220M    /var/tmp
120M    /var/spool
90M     /var/backups
60M     /var/mail
30M     /var/opt
10M     /var/local
```

This shows the largest directories in /var, sorted from biggest to smallest.

4 FIND SPECIFIC LARGE FILES WITH `find`

Use the `find` command to locate large files.

```
ubuntu@server:~$ find / -type f -size +100M 2>/dev/null
/var/log/app/app.log
/var/log/nginx/access.log
/var/lib/docker/containers/abc123-json.log
/var/www/html/backup.tar.gz
/home/ubuntu/largefile.iso
```

`-type f` – search for files
`-size +100M` – files larger than 100 MB
`2>/dev/null` – hide permission errors

Adjust the size (+100M, +1G, etc.) based on what you are looking for.

5 FILESYSTEM VS DIRECTORY USAGE

Filesystem usage (`df`)

- Shows overall disk usage for mounted filesystems
- Helps you see if a disk/partition is full
- Example: / (root), /var, /home

Directory usage (`du`)

- Shows the size of specific directories and files
- Helps you find exactly what is consuming space
- Use together: `df` to identify the full disk, `du` and `find` to locate the large files or folders

6 PROJECT: LOCATE WHAT IS CONSUMING DISK SPACE



- 1 Check disk usage with `df -h`
- 2 Identify the filesystem that is almost full
- 3 Use `du -sh` to find the largest directories
- 4 Use `find` to locate large files (e.g. > 100M)
- 5 Identify large log files (e.g. in /var/log)
- 6 Document what is consuming the space
- 7 Suggest a safe cleanup plan (e.g. rotate or compress logs)

Q IDENTIFY LARGE LOG FILES

Log files often grow very large and can fill up the disk.

```
ubuntu@server:~$ du -sh /var/log/* | sort -hr | head -5
1.8G    /var/log/app.log
300M    /var/log/nginx
120M    /var/log/syslog
90M     /var/log/auth.log
50M     /var/log/kern.log
```

Consider log rotation, compression, or archiving for old logs.



PRODUCTION WARNING

- Do not randomly delete files to recover space.
- Understand what the files are and why they are there.
- Deleting the wrong files can break applications, databases, or the entire system.
- Use proper log rotation and cleanup procedures.



JUNIOR ENGINEER TIP

Investigate first, then clean up safely. Disk space problems are symptoms. Find and fix the root cause (e.g. unrotated logs, runaway application, old backups).



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



CREATE A LINUX BACKUP SCRIPT



Learn
Practice
Build
Grow

Automate Backups. Protect Your Data. Build Real DevOps Skills.

1 BASH SCRIPT STRUCTURE

A simple Bash script has a shebang, comments and commands.

```
#!/bin/bash
# backup-app.sh
# This script backs up application
# config
# Author: VERIQTA

echo "Starting backup..."
```

The shebang (#!/bin/bash) informs the system to use Bash to run the script.

2 VARIABLES

Variables help make your script flexible and reusable.

```
APP_DIR="/etc/myapp"
BACKUP_DIR="/backup"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="myapp-backup-$DATE.tar.gz"
```

Key points:

- Use meaningful variable names
- Use \$(date ...) to create a timestamp
- Always quote your variables

3 CREATE A BACKUP DIRECTORY

Create a directory to store your backups.

```
sudo mkdir -p $BACKUP_DIR
echo "Backup directory created:"
echo $BACKUP_DIR
```

The -p option creates the directory if it does not exist and avoids errors if it already exists.

4 USE tar TO CREATE AN ARCHIVE

Use tar to package the files or directories you want to back up.

```
tar -cvf myapp-backup-$DATE.tar \
$APP_DIR
```

- c create archive
- v verbose (show files)
- f specify file name

5 COMPRESS WITH gzip

Use gzip to compress the archive and save disk space.

```
tar -cvf - $APP_DIR | \
gzip > $BACKUP_DIR/$BACKUP_FILE
```

This creates a compressed .tar.gz file directly, without keeping an uncompressed archive.

6 USE TIMESTAMPS

Timestamps make each backup unique and help with versioning.

```
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="myapp-backup-$DATE.tar.gz"
```

Example output:

myapp-backup-2025-08-27_14-30-05.tar.gz

Format: YYYY-MM-DD_HH-MM-SS

7 BACK UP APPLICATION CONFIGURATION

Now put it together. This example backs up a sample application configuration directory.

```
tar -cvf - $APP_DIR | \
gzip > $BACKUP_DIR/$BACKUP_FILE

echo "Backup completed: $BACKUP_FILE"
```

8 VERIFY THE ARCHIVE

Check that the backup file was created and contains the expected files.

```
ls -lh $BACKUP_DIR
tar -tzf $BACKUP_DIR/$BACKUP_FILE
```

What to check:

- File exists
- File size looks correct
- You can list the contents
- Important configuration files are included

9 PROJECT OUTPUT

Save the script as backup-app.sh.

```
nano backup-app.sh
chmod +x backup-app.sh
./backup-app.sh
```

Final script name:

backup-app.sh

A reusable script you can schedule with cron later.

COMPLETE SCRIPT EXAMPLE

```
#!/bin/bash
# backup-app.sh
# Backs up application configuration
APP_DIR="/etc/myapp"
BACKUP_DIR="/backup"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="myapp-backup-$DATE.tar.gz"
echo "Creating backup directory..."
sudo mkdir -p $BACKUP_DIR
echo "Backing up $APP_DIR ..."
tar -cvf - $APP_DIR | gzip > $BACKUP_DIR/$BACKUP_FILE
echo "Backup completed: $BACKUP_DIR/$BACKUP_FILE"
```



JUNIOR ENGINEER TIP

- ✓ Use clear and descriptive file names.
- ✓ Always test your backup and verify the contents.
- ✓ Consider automating this with cron.
- ✓ Store backups in a separate disk or cloud storage for disaster recovery.
- ✓ Keep only the backups you need (e.g. last 7 or 30 days).



IMPORTANT WARNING

- ✗ Do not randomly delete files to recover disk space.
- ✗ Understand what you are backing up and why.
- ✗ Verify your backups regularly.
- ✗ Consider file permissions and sensitive data.
- ✗ In production, follow your organization's backup policy.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



AUTOMATE BACKUPS WITH CRON



Schedule Tasks. Save Time. Keep Your Systems Safe.

1 WHAT CRON SOLVES

Cron is a time-based job scheduler in Linux. It runs commands automatically at scheduled times.

- ✓ Runs repetitive tasks without manual work
- ✓ Useful for backups, log rotation, updates, etc.
- ✓ Helps you automate routine operations
- ✓ Runs in the background

3 CRON SYNTAX

A cron job has 5 time fields followed by the command.

* * * * * `command_to_run`

1st field	Minute	(0 - 59)
2nd field	Hour	(0 - 23)
3rd field	Day of month	(1 - 31)
4th field	Month	(1 - 12)
5th field	Day of week	(0 - 7) (0 or 7 = Sunday)

EXAMPLES

- 0 2 * * * Run at 2:00 AM every day
- 0 2 * * 0 Run at 2:00 AM every Sunday
- 30 1 * * * Run at 1:30 AM every day

2 OPEN CRONTAB

Use `crontab -e` to create or edit your cron jobs for the current user.

```
crontab -e
```

This opens a file where you add scheduled tasks.

4 SCHEDULE THE PAGE 11 BACKUP

From Page 11, we created a backup script (e.g. `/home/ubuntu/linux-labs/scripts/backup-app.sh`). Now schedule it to run daily at 2:00 AM.

```
0 2 * * * /home/ubuntu/linux-labs/scripts/backup-app.sh
```

5 REDIRECT OUTPUT TO LOGS

Always redirect output so you can review what happened.

```
0 2 * * * /home/ubuntu/linux-labs/scripts/backup-app.sh \
>> /home/ubuntu/linux-labs/logs/backup.log 2>&1
```

6 VERIFY EXECUTION

After the scheduled time, check the log file.

```
cat /home/ubuntu/linux-labs/logs/backup.log
```

You should see the output from your backup script.

7 TROUBLESHOOT FAILED CRON JOBS

- ❗ Check if the script exists and is executable
`ls -l /home/ubuntu/linux-labs/scripts/backup-app.sh`
- ❗ Check cron logs
`grep CRON /var/log/syslog # (Ubuntu/Debian)`
- ❗ Verify the user's crontab
`crontab -l`
- ❗ Check the script manually
`bash /home/ubuntu/linux-labs/scripts/backup-app.sh`

8 PRODUCTION CONNECTION



AUTOMATION MUST ALSO BE OBSERVABLE.

- ✓ Always log output
- ✓ Monitor for failures
- ✓ Set up alerts for critical jobs
- ✓ Keep scripts simple and reliable
- ✓ Test your backups regularly
- ✓ A backup that is not tested is not a backup



JUNIOR ENGINEER TIP

Automate the boring tasks, but always verify that they run successfully.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



VERIQTA



BUILD AND MANAGE A systemd SERVICE



Learn
Build
Practice
Grow

Create, Control and Make Your Application Start Automatically

1 WHAT SYSTEMD DOES

systemd is the init system and service manager used by most modern Linux distributions. It starts, stops, and manages services, handles dependencies, and ensures applications run reliably at boot.

- ✓ Starts services at boot
- ✓ Keeps services running
- ✓ Manages dependencies
- ✓ Provides logging and monitoring
- ✓ Replaces older init systems (like SysVinit)

2 CREATE A SIMPLE APPLICATION

We will create a simple script that outputs a message every 10 seconds.

```
sudo mkdir -p /opt/myapp
sudo nano /opt/myapp/app.sh

#!/bin/bash
while true; do
    echo "$(date): Hello from VERIQTA systemd service"
    >> /opt/myapp/app.log
    sleep 10
done
```

Make it executable:

```
sudo chmod +x /opt/myapp/app.sh
```

Simple
App
Ready!

3 CREATE A .SERVICE UNIT

Create a systemd service file for your application.

```
sudo nano /etc/systemd/system/myapp.service

[Unit]
Description=My Sample Application Service
After=network.target

[Service]
ExecStart=/opt/myapp/app.sh
Restart=always
User=root

[Install]
WantedBy=multi-user.target
```

4 MANAGE THE SERVICE

Use systemctl to control your service.

systemctl start myapp	Start the service
systemctl stop myapp	Stop the service
systemctl restart myapp	Restart the service
systemctl status myapp	Check the service status
systemctl enable myapp	Enable at boot (start automatically)
systemctl disable myapp	Disable at boot
systemctl is-enabled myapp	Check if enabled

5 RELOAD AND VERIFY

After creating or editing the service file, reload systemd and start your service.

```
sudo systemctl daemon-reload
sudo systemctl start myapp
sudo systemctl status myapp
```

6 CHECK THE LOGS

View your application logs and systemd logs.

```
tail -f /opt/myapp/app.log
journalctl -u myapp -f
```

7 PROJECT GOAL



Make your application start automatically on boot and keep running in the background.



JUNIOR ENGINEER TIP

Use systemctl status and logs to troubleshoot issues. A service may be "active" but your application could still have errors. Always verify the actual output.

"AUTOMATION
TURNS GOOD
ENGINEERS INTO
GREAT ENGINEERS."



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



VERIQTA

KNOWLEDGE | PRACTICE | OPPORTUNITY



TROUBLESHOOT A FAILED LINUX SERVICE

FIND THE PROBLEM. FIX IT. KEEP IT RUNNING.



Learn
Build
Practice
Grow

1 DELIBERATELY BREAK THE SERVICE

Let's introduce an error in the service we created on Page 13. Edit the service file and use a wrong path.

```
sudo nano /etc/systemd/system/myapp.service
# Change ExecStart to an invalid path
ExecStart=/opt/myapp/wrong-app.sh
```

! This will cause the service to fail.

3 USE JOURNALCTL

View detailed logs for the service.

```
sudo journalctl -u myapp -n 20
```

```
Jun 10 10:15:32 server systemd[1]: Starting My Sample App...
Jun 10 10:15:32 server myapp[1234]: /bin/bash:
/opt/myapp/wrong-app.sh: No such file or directory
Jun 10 10:15:32 server systemd[1]: myapp.service:
Main process exited, code=exited, status=203/EXEC
Jun 10 10:15:32 server systemd[1]: Failed to start myapp.
```

5 IDENTIFY THE ACTUAL ERROR

From the logs, the error is clear:

/opt/myapp/wrong-app.sh: No such file or directory

- The service is trying to run a script that does not exist.
- This is the real cause, not a systemd issue.

7 RESTART THE SERVICE

Reload systemd and restart the service.

```
sudo systemctl daemon-reload
sudo systemctl restart myapp
sudo systemctl status myapp
```

8 VERIFY RECOVERY

Check that the service is now running.

```
sudo systemctl status myapp
```

```
● Active: active (running) since Mon 2024-06-10 10:20:11
Main PID: 4567 (app.sh)
Tasks: 1 (limit: 4915)
Memory: 5.2M
```

2 READ SYSTEMCTL STATUS

Check the service status to see the failure.

```
sudo systemctl status myapp
```

```
● myapp.service - My Sample Application Service
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
   Active: failed (Result: exit-code) since Mon 2024-06-10 10:15:32
   Process: 1234 ExecStart=/opt/myapp/wrong-app.sh (code=exited,
   status=203/EXEC)
   Failed to start myapp.service.
```

4 INSPECT SERVICE LOGS

You can also view logs in real time.

```
sudo journalctl -u myapp -f
```

```
Jun 10 10:16:01 server systemd[1]: myapp.service:
Failed with result 'exit-code'.
Jun 10 10:16:01 server systemd[1]: Failed to start
My Sample Application Service.
```

6 CORRECT THE CONFIGURATION

Edit the service file and set the correct path.

```
sudo nano /etc/systemd/system/myapp.service
ExecStart=/opt/myapp/app.sh # correct path
```

9 KEY LESSON: TROUBLESHOOTING FLOW

- Symptom** (Service not running)
- Evidence** (Check status and logs)
- Cause** (Find the real error)
- Fix** (Correct the configuration)
- Verify** (Confirm it is working)



PROJECT TAKEAWAY

You intentionally broke the service, investigated the issue using real tools, fixed it, and restored it. This is a real-world troubleshooting workflow used by DevOps and SRE engineers.

"PROBLEMS ARE OPPORTUNITIES TO BECOME A BETTER ENGINEER."



@veriqta



veriqta



veriqta



veriqta



@veriqta

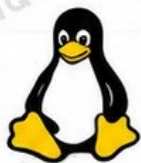


@veriqta



VERIQTA

KNOWLEDGE | PRACTICE | OPPORTUNITY



INVESTIGATE LINUX NETWORK CONNECTIVITY



Learn
Build
Practice
Grow

FIND THE ISSUE. KNOW THE LAYER. FIX THE PROBLEM.

1 CHECK NETWORK INTERFACES (IP ADDRESS)

View your server's network interfaces and IP addresses.

```
ip addr
```

```
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500  
inet 192.168.1.10/24 brd 192.168.1.255  
inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
```

- ✓ Shows your IP address, network interface status and details.

2 CHECK ROUTING (GATEWAY)

View your routing table.

```
ip route
```

```
default via 192.168.1.1 dev ens3  
192.168.1.0/24 dev ens3 proto kernel  
192.168.1.1 dev ens3 proto kernel
```

- ✓ Shows your default gateway and network routes.
- ✓ Helps you know where traffic is going.

3 TEST CONNECTIVITY WITH PING

Check if a host is reachable.

```
ping -c 4 google.com
```

```
PING google.com (142.250.72.14)  
56(84) bytes of data:  
64 bytes from 142.250.72.14:  
icmp_seq=1 ttl=117 time=12.3 ms  
...  
--- google.com ping statistics ---  
4 packets transmitted, 4 received  
0% packet loss
```

- ✓ Tests basic network connectivity (ICMP).

4 TEST APPLICATION ACCESS WITH CURL

Check if an application (HTTP/HTTPS) is reachable.

```
curl -I https://google.com
```

```
HTTP/2 200  
content-type: text/html; charset=UTF-8  
content-length: 172344  
...
```

- ✓ Tests real application connectivity.
- ✓ Useful for APIs and web services.

5 CHECK LISTENING PORTS WITH SS

View open ports and listening services.

```
ss -tulnp
```

Netid	State	Local Address:Port
tcp	LISTEN	0.0.0.0:22
tcp	LISTEN	127.0.0.1:8080
udp	LISTEN	0.0.0.0:53

- ✓ Shows which ports are listening.
- ✓ Helps confirm if a service is running and reachable.

6 DNS LOOKUP BASICS

Resolve a domain name to an IP address.

```
nslookup google.com
```

```
Server: 192.168.1.1  
Address: 192.168.1.1#53  
  
Non-authoritative answer:  
Name: google.com  
Address: 142.250.72.14
```

- ✓ Checks DNS resolution.
- ✓ Use dig google.com for more details (if installed).

7 PROJECT: DIAGNOSE A CONNECTION ISSUE

Your server cannot reach an application. Use the tools above to find and fix the problem.

EXAMPLE SCENARIO

Your server cannot access <https://myapp.company.com>

- 1 Check DNS resolution (nslookup or dig)
- 2 Check network connectivity (ping)
- 3 Check routing (ip route)
- 4 Check if the port is open (ss or curl)
- 5 Identify the real issue (DNS, routing, port or application)
- 6 Fix the issue and verify



8 COMMON FAILURES AND HOW TO IDENTIFY THEM

Failure Type	Symptom	How to Check
DNS Failure	Cannot resolve domain name	nslookup dig
Routing Failure	Host not reachable	ip route ping (gateway)
Port Failure	Connection refused or timed out	ss -tulnp curl
Application Failure	Port open but service not working	curl (check response)

9 KEY TAKEAWAYS

- ✓ Use the right tool for the right layer.
- ✓ DNS must resolve first.
- ✓ Routing must be correct.
- ✓ The port must be open.
- ✓ The application must be healthy.
- ✓ Network troubleshooting is a layered process.



JUNIOR ENGINEER TIP

When troubleshooting, always test from the server itself. A problem could be local to the server, the network, DNS, a firewall, or the application. Check each layer step by step.

"DIAGNOSE
DON'T GUESS."
— VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



VERIQTA



FIND WHAT IS LISTENING ON A PORT



Learn
Build
Practice
Grow

DISCOVER. VERIFY. TROUBLESHOOT. TAKE CONTROL.

1 IP ADDRESSES VS PORTS

An IP address identifies a machine.
A port identifies a service on that machine.

- IP address = server (e.g. 192.168.1.10)
- Port = specific application (e.g. 80, 443)
- Same server can have many services, each on a different port.
- Example: 192.168.1.10:80 (HTTP)
192.168.1.10:443 (HTTPS)

2 LISTENING SOCKETS

A listening socket means a service is waiting for incoming connections on a specific port.

- ✓ If a service is listening, it will show in the socket list.
- ✓ If nothing is listening, connections will be refused.
- ✓ A service can listen on all interfaces (0.0.0.0) or a specific IP (127.0.0.1).

3 USE ss -tulpn

The ss command shows listening ports and the processes that own them.

ss -tulpn

Netid	State	Local Address:Port	Process
tcp	LISTEN	0.0.0.0:22	users:((("sshd",pid=722,fd=3))
tcp	LISTEN	0.0.0.0:80	users:((("nginx",pid=1234,fd=6))
tcp	LISTEN	127.0.0.1:5432	users:((("postgres",pid=567,fd=7))

- ✓ -t = TCP -u = UDP -l = listening
- ✓ -p = show process -n = show numbers
- ✓ Shows which process owns each port.

4 TEST A PORT WITH curl

Use curl to test if a service is reachable.

curl -I http://localhost:80

```
HTTP/1.1 200 OK
Server: nginx/1.24.0
Date: Mon, 10 Jun 2024 10:00:00 GMT
Content-Type: text/html
```

- ✓ Confirms the service responds.
- ✓ Useful for HTTP/HTTPS and APIs.
- ✓ For non-HTTP services, use tools like nc or telnet.

5 PROCESS-TO-PORT RELATIONSHIP

Find which process is using a specific port.

sudo lsof -i :80

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
nginx	1234	root	6u	IPv4	12345	0t0	TCP	*:80 (LISTEN)

- ✓ Shows the process name and PID.
- ✓ Helps you identify and manage the service.
- ✓ You can also use: **ps -p <PID>** to see more details.

6 PROJECT: IDENTIFY WHICH SERVICE OWNS A PORT

- List all listening ports
ss -tulpn
- Find the service using a specific port (e.g. 80, 443, 8080)
- Confirm the service is responding with curl
- Identify the process and verify it matches the expected service



PROJECT GOAL

Identify which service owns a port and confirm it is running and reachable.

7 FAILURE EXERCISE: INVESTIGATE "CONNECTION REFUSED"

A connection refused error means nothing is listening on the target port.

```
$ curl http://localhost:8080
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

- Check if the service is running
- Verify it is listening on the correct port
- Confirm firewall is not blocking the port
- Check the service logs
- Start or fix the service, then test again

8 COMMON CAUSES AND FIXES

Cause	Symptom	Fix
Service not running	Connection refused	Start the service (systemctl start)
Wrong port	Connection refused	Check the correct port (configuration)
Service listening on localhost only	Cannot reach from other servers	Configure to listen on 0.0.0.0 or server IP
Firewall blocking port	Connection times out	Open the port (ufw/iptables)
Wrong service	Unexpected response	Verify the process using the port
Application error	Connection reset or 5xx error	Check application logs

9 KEY TAKEAWAYS

- ✓ An IP address identifies the server, a port identifies the service.
- ✓ Use **ss -tulpn** to see listening ports.
- ✓ Use **curl** to test connectivity.
- ✓ You can find the process that owns a port with **lsof**.
- ✓ Connection refused means nothing is listening.
- ✓ Always check the service, firewall and configuration.
- ✓ Network troubleshooting is a key DevOps and SRE skill.



JUNIOR ENGINEER TIP

When a connection fails, don't guess. Check each layer: server → port → service → application.

"CLEAR VISIBILITY
PREVENTS DOWNTIME."
— VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



DEPLOY A SIMPLE WEB SERVER

Install. Configure. Serve. Verify. You're Live!

1 INSTALL NGINX (OR ANOTHER WEB SERVER)

We will use NGINX. It is lightweight, fast and widely used in production.



Ubuntu / Debian

```
sudo apt update
sudo apt install nginx -y
```



Amazon Linux / RHEL / CentOS

```
sudo yum install nginx -y
# or on newer systems
sudo dnf install nginx -y
```

4 CREATE A BASIC HTML PAGE

Replace the default page with your own.

```
sudo bash -c 'cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
<title>Welcome to VERIQTA</title>
</head>
<body>
<h1>VERIQTA Linux Project</h1>
<p>My first web server is running!</p>
</body>
</html>
EOF'
```



The web root for NGINX is /var/www/html
The main file is index.html

6 INSPECT LISTENING PORTS

Check which port NGINX is using (default is 80).

```
sudo ss -tulpn | grep :80
# or
sudo netstat -tulpn | grep :80
```



You should see
nginx listening
on 0.0.0.0:80
or [::]:80

2 START AND ENABLE THE SERVICE

Start NGINX and enable it to start at boot.

```
sudo systemctl start nginx
sudo systemctl enable nginx
# Check if it started
sudo systemctl is-active nginx
```

3 INSPECT SERVICE STATUS

Verify the service is running.

```
sudo systemctl status nginx
# Example output: active (running)
```



If you see
active (running),
NGINX is working
correctly.

5 TEST WITH CURL

Test the web server from your server or another machine.

```
curl http://localhost
# You should see your HTML content
# Example:
<h1>VERIQTA Linux Project</h1>
<p>My first web server is running!</p>
```



You can also test using your server's IP address
from another machine in your network or browser.

7 READ ACCESS AND ERROR LOGS

NGINX logs help you troubleshoot issues.

Access log (requests)

```
sudo tail -f /var/log/nginx/
access.log
```

Error log (errors)

```
sudo tail -f /var/log/nginx/
error.log
```

8 PROJECT RESULT



A working Linux web server!

You have installed, configured, and deployed NGINX,
created a web page, tested it, and checked the logs.
This is a real-world skill used in production.



TROUBLESHOOTING TIPS

- If the page does not load, check firewall settings.
- Ensure the service is running: `sudo systemctl status nginx`
- Check error logs: `sudo tail -f /var/log/nginx/error.log`



JUNIOR ENGINEER TIP

A web server is just the beginning. The same steps
apply to other services you will deploy as a DevOps
or Platform Engineer.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



BUILD A SERVER HEALTH-CHECK SCRIPT



MONITOR. AUTOMATE. STAY AHEAD.

1 WHAT WE ARE BUILDING

A simple shell script that checks the health of your server and logs the results.

- ✓ Check disk usage
- ✓ Check memory
- ✓ Check uptime
- ✓ Check a service
- ✓ Check a listening port
- ✓ Write results to a log
- ✓ Use exit codes

2 CREATE THE SCRIPT FILE

Create a file named health-check.sh in your linux-labs directory.

```
cd ~/linux-labs/scripts
touch health-check.sh
chmod +x health-check.sh
```



Make the script executable so you can run it.

```
chmod +x health-check.sh
```

4 RUN THE SCRIPT

Execute the script to see the output.

```
./health-check.sh
```

The results will be displayed in the terminal and also saved to the log file.

5 CHECK THE LOG FILE

View the saved results.

```
cat /var/log/health-check.log
```



Each time you run the script, new results are appended to the log file.

6 UNDERSTAND EXIT CODES

The script uses exit codes to indicate success or failure.

Exit Code	Meaning
0	All checks passed (healthy)
1	Service or port is not running (unhealthy)



Exit codes allow this script to be used in automation, monitoring tools, or cron jobs.

7 EXAMPLE OUTPUT

Sample output from a healthy server.

```
===== Server Health Check =====
Date: Mon Aug 26 10:00:00 UTC 2026
-----
1. Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   12G   36G   25% /
2. Memory Usage:
              total      used      free     shared  buff/cache   available
Mem:          1.1G    512M    100M        0M       400M       700M
Swap:          0B         0B         0B
3. Uptime:
10:00:00 up 2 days, 3:20, 1 user, load average: 0.05, 0.03, 0.01
4. Service Status (nginx):
nginx is running
5. Check Port 80:
Port 80 is open
-----
Health check completed
```

3 THE SCRIPT

Edit the file and add the following code.

```
#!/bin/bash
# health-check.sh
# Simple server health check script
# Logs system health and uses exit codes

LOG_FILE="/var/log/health-check.log"
SERVICE="nginx"
PORT=80

echo "===== Server Health Check =====" | tee -a $LOG_FILE
echo "Date: $(date)" | tee -a $LOG_FILE
echo "-----" | tee -a $LOG_FILE

# Check disk usage
echo "1. Disk Usage:" | tee -a $LOG_FILE
df -h | tee -a $LOG_FILE

# Check memory
echo "2. Memory Usage:" | tee -a $LOG_FILE
free -h | tee -a $LOG_FILE

# Check uptime
echo "3. Uptime:" | tee -a $LOG_FILE
uptime | tee -a $LOG_FILE

# Check service status
echo "4. Service Status ($SERVICE):" | tee -a $LOG_FILE
if systemctl is-active --quiet $SERVICE; then
    echo "$SERVICE is running" | tee -a $LOG_FILE
else
    echo "WARNING: $SERVICE is NOT running" | tee -a $LOG_FILE
fi

# Check listening port
echo "5. Check Port $PORT:" | tee -a $LOG_FILE
if ss -tln | grep -q ":$PORT"; then
    echo "Port $PORT is open" | tee -a $LOG_FILE
else
    echo "WARNING: Port $PORT is NOT open" | tee -a $LOG_FILE
fi

echo "-----" | tee -a $LOG_FILE
echo "Health check completed" | tee -a $LOG_FILE

# Exit with non-zero code if service or port is down
if ! systemctl is-active --quiet $SERVICE || \
    ! ss -tln | grep -q ":$PORT"; then
    exit 1
else
    exit 0
fi
```

8 DEVOPS CONNECTION

This script introduces key DevOps concepts:

- Automation (repeatable checks)
- Observability (logs and exit codes)
- Monitoring (can be integrated with tools)
- Proactive operations (detect issues early)
- Can be scheduled with cron for continuous monitoring



PROJECT RESULT



health-check.sh

A working Linux server health-check script that you can extend and use in real environments.



NEXT STEPS

- Add more checks (CPU, disk per partition, etc.)
- Send alerts (email, Slack, etc.)
- Use with cron for regular monitoring
- Integrate with monitoring tools (Prometheus, Zabbix, etc.)



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



VERIQTA



TROUBLESHOOT A BROKEN LINUX SERVER

FIND THE PROBLEM. THINK LIKE AN ENGINEER. FIX IT.



Learn
Build
Practice
Grow

1 SCENARIO

Your Linux server is not working as expected. Several issues have been introduced. Your task is to investigate, find the root causes, and fix them.



GOAL

Restore the server to full working condition without being given step-by-step solutions. Use your Linux skills, problem-solving ability, and the troubleshooting workflow.

3 TROUBLESHOOTING WORKFLOW

Use a structured approach to solve the problems.



4 USEFUL COMMANDS (HINTS)

These commands may help you during your investigation.

<code>systemctl status <service></code>	Check service status
<code>systemctl start <service></code>	Start a service
<code>ls -l</code>	Check file permissions
<code>cat <file></code>	View file content
<code>grep <pattern> <file></code>	Search in files
<code>ss -tulpn</code>	Check listening ports
<code>journalctl -u <service></code>	View service logs
<code>tail -f <logfile></code>	Monitor logs in real time
<code>df -h</code>	Check disk space
<code>free -h</code>	Check memory usage



These are hints, not complete solutions. Explore and use other commands where needed.



CHALLENGE YOURSELF

- Do not rush to fix everything.
- Understand the root cause.
- Try to solve each problem without external help.
- This is how real engineers work in production.

2 INTRODUCED FAILURES

The server has the following issues (deliberately introduced):

- Wrong file permissions**
A critical file or directory has incorrect permissions.
- Stopped service**
A required service (e.g. NGINX) is not running.
- Incorrect configuration**
A configuration file contains an error.
- Missing file**
A required file is missing.
- Port not listening**
The application is not listening on the expected port.
- Application unavailable**
The web application is not accessible.

5 INVESTIGATE AND FIX

Work through each issue, one by one.

- 1 Identify the actual problem (not just the symptom).
- 2 Gather evidence using the right commands.
- 3 Decide on the most likely cause.
- 4 Test your hypothesis.
- 5 Apply the fix.
- 6 Verify that the issue is resolved.

6 PROJECT RESULT



A fully working Linux server!

All services running, correct configurations, proper permissions, application accessible, and no errors in logs.

7 KEY TAKEAWAYS

- ✓ Think and troubleshoot like a real engineer.
- ✓ Learn to read logs and error messages.
- ✓ Use a structured workflow.
- ✓ Develop confidence in solving real-world problems.
- ✓ These skills are essential for DevOps, SRE and Linux engineers.



JUNIOR ENGINEER TIP

In real environments, multiple issues can happen at the same time. Stay calm, investigate systematically, and focus on facts, not assumptions.

FINAL PROJECT >

OPERATE A MINI PRODUCTION LINUX SERVER

PUT EVERYTHING TOGETHER. THINK. TROUBLESHOOT. RUN IT LIKE A PRO.

1 PROJECT OVERVIEW

In this final project, you will set up, configure and operate a mini production Linux server from scratch. You will apply everything you have learned in the previous pages. This is a realistic hands-on scenario that simulates what Linux and DevOps engineers do in real environments.

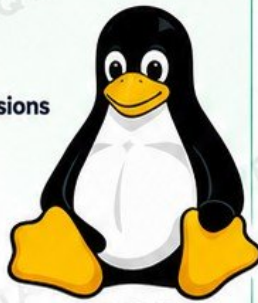


GOAL

Build a fully working Linux server that runs a web application, is secure, observable, automated and can be recovered after a failure.

2 PROJECT TASKS

- 1 Create users and groups
- 2 Build application directories
- 3 Apply secure ownership and permissions
- 4 Deploy the web application
- 5 Configure a systemd service
- 6 Verify network connectivity
- 7 Inspect ports
- 8 Configure logs
- 9 Build the backup script
- 10 Schedule backups
- 11 Build the health-check script
- 12 Introduce a deliberate failure
- 13 Diagnose it
- 14 Restore service
- 15 Verify the application
- 16 Document what happened



LINUX
POWERS
POSSIBILITIES

"PRACTICE
BUILDS
REAL
ENGINEERS."

4 DELIVERABLES



- ✓ Working web application
- ✓ Configured systemd service
- ✓ Secure users, groups and permissions
- ✓ Backup script with scheduled job
- ✓ Health-check script
- ✓ Logs and documentation
- ✓ Tested failure and successful recovery

5 TROUBLESHOOTING TIPS

- ! Use logs (systemctl, journalctl, /var/log).
- ! Check service status and configuration.
- ! Verify network connectivity and ports.
- ! Look for permission and file issues.
- ! Be methodical and patient.
- ! Use the troubleshooting workflow (Observe → Evidence → Hypothesis → Test → Fix → Verify).

3 FINAL PROJECT MENTAL MODEL



Same
Skills.
Real
Impact.



PROBLEMS
WILL HAPPEN.

YOUR ABILITY
TO INVESTIGATE
AND RECOVER
MAKES YOU A
GREAT ENGINEER.



JUNIOR ENGINEER TIP

This project is not about doing everything perfectly the first time. It is about thinking like an engineer, solving problems and building confidence.



A SMALL SERVER TODAY.
A BIGGER FUTURE TOMORROW.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta





FINAL PROJECT

OPERATE A MINI PRODUCTION

LINUX SERVER



Learn
Build
Practice
Grow

BRING EVERYTHING TOGETHER. BUILD. OPERATE. TROUBLESHOOT. SUCCEED.

1 CREATE USERS AND GROUPS



- Create a dedicated user for the application
- Create a group and add the user
- Verify with id and groups

2 BUILD APPLICATION DIRECTORIES



- Create a structured directory layout
- e.g. /opt/myapp, /var/log/myapp, /etc/myapp
- Organize files properly

3 APPLY SECURE OWNERSHIP AND PERMISSIONS



- Set correct ownership (chown)
- Apply secure permissions (chmod)
- Follow the principle of least privilege

4 DEPLOY THE WEB APPLICATION



- Install and configure NGINX (or your application)
- Deploy a simple web page
- Test that it works

5 CONFIGURE A SYSTEMD SERVICE



- Create a systemd service file
- Start, enable, and verify the service
- Ensure it restarts on reboot

6 VERIFY NETWORK CONNECTIVITY



- Check local and external connectivity
- Use ping, curl and ip tools
- Confirm the application is reachable

7 INSPECT PORTS



- Check which ports are listening
- Use ss or netstat
- Verify the web server port (e.g. 80)

8 CONFIGURE LOGS



- Verify access and error logs
- Check log file locations
- Generate some test logs

9 BUILD THE BACKUP SCRIPT



- Create a script to back up application files
- Include timestamps
- Test the script manually

10 SCHEDULE BACKUPS



- Add the backup script to cron
- Verify the schedule
- Check that backups are created

11 BUILD THE HEALTH-CHECK SCRIPT



- Create a script to check server health
- Check disk, memory, service and port
- Use exit codes
- Test the script

12 INTRODUCE A DELIBERATE FAILURE



- Break something intentionally
- e.g. stop the service, change permissions, remove a file or wrong config.

13 DIAGNOSE IT



- Use your Linux skills to investigate
- Check logs, service status, configs
- Identify the root cause

14 RESTORE SERVICE



- Apply the correct fix
- Start the service (if needed)
- Verify everything is working

15 VERIFY THE APPLICATION



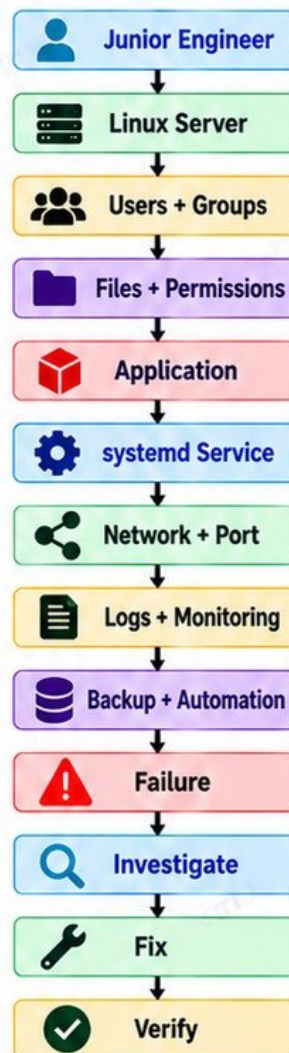
- Confirm the application is accessible
- Test with curl or a browser
- Check logs and service status

16 DOCUMENT WHAT HAPPENED



- Write a short summary
- What failed and
- How you fixed it
- What you learned

FINAL PROJECT MENTAL MODEL



Same
Skills
Real
Servers
Real
Careers

A SMALL
PROJECT
TODAY
A BIGGER
OPPORTUNITY
TOMORROW.

KEY TAKEAWAYS

- ✓ You used real Linux skills.
- ✓ You operated a complete workflow.
- ✓ You practiced troubleshooting.
- ✓ You learned how systems fit together.
- ✓ This is what DevOps, SRE and infrastructure engineers do in real environments.

★ **JUNIOR ENGINEER TIP**
Perfection is not the goal.
Learning, practice, and
problem-solving are.

**"REAL ENGINEERS
TURN PROBLEMS
INTO PROGRESS."**

— VERIQTA



PROJECT RESULT

A working Linux server with a deployed application, automated backups, monitoring script, and the ability to troubleshoot and recover from failures.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



VERIQTA

KNOWLEDGE | PRACTICE | OPPORTUNITY